DATA ENGINEERING PROJECT — COMPLETE TECHNICAL REPORT

Retail Analytics Lakehouse

End-to-End Medallion Architecture on Databricks

Bronze → Silver → Gold → Power BI

Platform: Databricks — Unity Catalog, Delta Lake, SQL Warehouses **Language:** SQL (Spark SQL / Databricks SQL) **Visualisation:** Microsoft Power BI	**Architecture:** Medallion — Bronze / Silver / Gold **Storage:** Unity Catalog Volumes (replaces DBFS) **Date:** March 2026

200,000	10,000	1,000	10
Raw Sales Records	Customers Processed	Products Catalogued	Delta Tables Created

Internship Project · Data Engineering Track · Databricks + Power BI · March 2026

1. Executive Summary

This report documents the complete implementation of the Retail Analytics Lakehouse — an end-to-end data engineering solution built on Databricks using Delta Lake, Unity Catalog, and SQL. The project transforms raw retail CSV files through a three-layer Medallion Architecture (Bronze, Silver, Gold) and delivers business-ready insights via a four-page Power BI dashboard.

The project was delivered across three weeks covering data ingestion, data quality engineering, dimensional modelling, business aggregation, automated orchestration, and visual analytics. Every component is production-grade — all tables are governed by Unity Catalog, all transformations are idempotent SQL, and every pipeline run is tracked in an operational audit log.

Deliverable	Status	Artefacts / Tables
Unity Catalog Setup	✓ **Complete**	retail_catalog.retail_analytics schema + Volume
Bronze Layer	✓ **Complete**	bronze_sales (200K rows), bronze_master (11,009 rows)
Silver Layer	✓ **Complete**	silver_customers, silver_products, silver_sales, dim_date
Data Quality Framework	✓ **Complete**	rejected_rows (15 records quarantined)
Fact & Dimension Models	✓ **Complete**	dim_customers, dim_products, fact_sales (star schema)
Gold Layer	✓ **Complete**	gold_daily_sales, gold_customer_ltv, gold_category_performance, gold_repeat_customer_rate
Performance Optimisation	✓ **Complete**	OPTIMIZE + ZORDER + VACUUM + ANALYZE + Partitioning
Automated Workflow	✓ **Complete**	3-task Databricks Job, CRON 0 0 2 * * ?
Audit Logging	✓ **Complete**	pipeline_audit_log — 10 tables, all ✓ SUCCESS
Power BI Dashboard	✓ **Complete**	4 report pages, DAX measures, star schema model, scheduled refresh

2. Architecture Overview

2.1 System Architecture Diagram

The diagram below illustrates the complete end-to-end data flow of the Retail Analytics Lakehouse — from the operational sales system through source tables, processing steps, warehouse schema, and finally to reports and analytics.

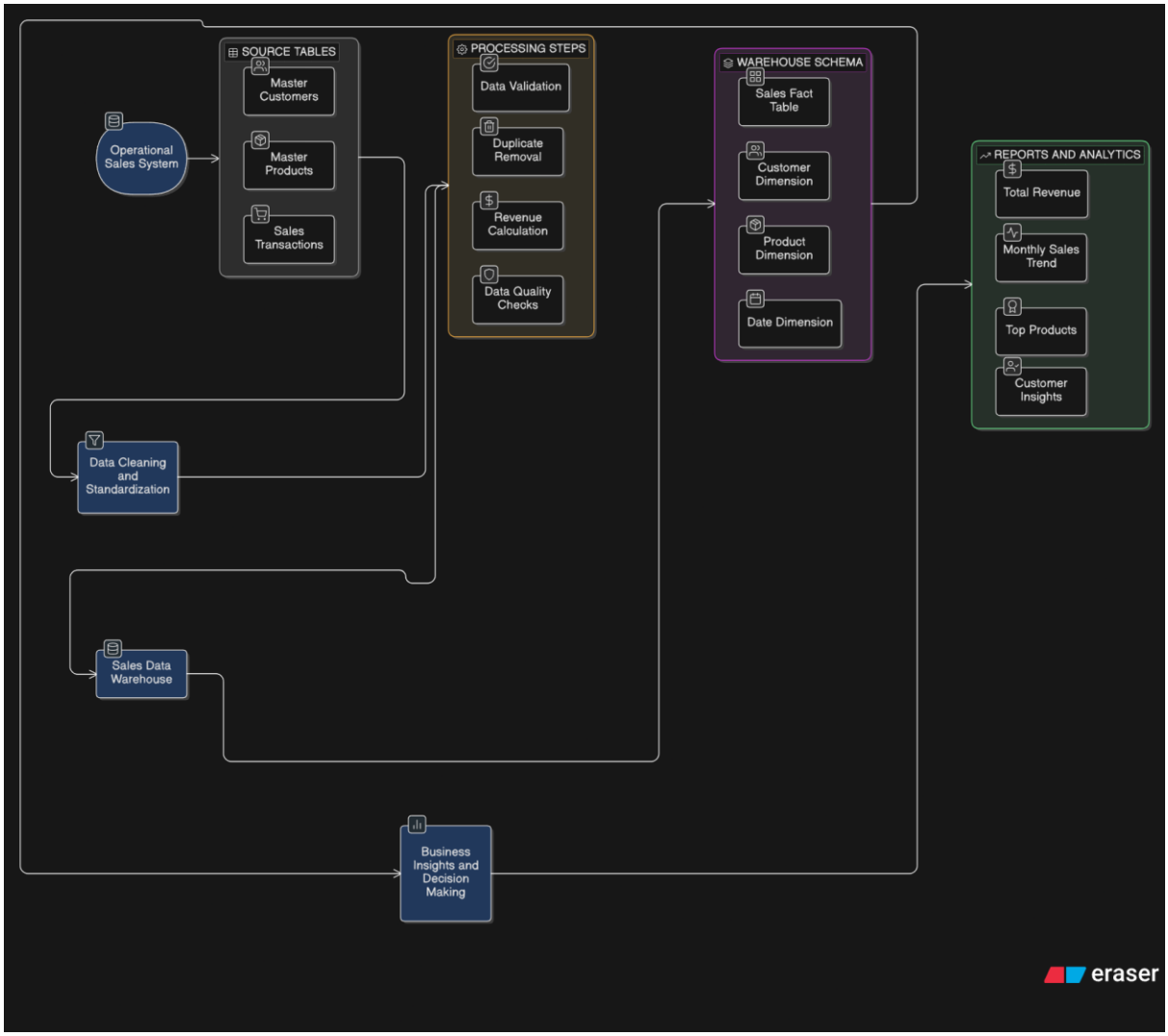

DIAGRAM WALKTHROUGH	Data flows left to right: Operational Sales System → Source Tables (Master Customers, Master Products, Sales Transactions) → Processing Steps (Data Validation, Duplicate Removal, Revenue Calculation, DQ Checks) → Warehouse Schema (Sales Fact Table + 3 Dimensions) → Reports & Analytics (Total Revenue, Monthly Sales Trend, Top Products, Customer Insights). The Sales Data Warehouse and Data Cleaning nodes represent the Silver layer processing.

2.2 Medallion Architecture Layers

The project implements the industry-standard Medallion (multi-hop) Lakehouse architecture. Each layer serves a distinct purpose and quality level, ensuring raw data is always preserved and business users consume only clean, aggregated data.

Layer	Storage Location	Tables	Row Count	Purpose
Volume (Landing)	/Volumes/retail_catalog/retail_analytics/retail_data/	3 CSV files	—	Raw file landing zone — no transformation
Bronze	retail_catalog.retail_analytics.*	bronze_sales, bronze_master	200,000 + 11,009	Exact CSV replica as Delta — all STRING, metadata appended
Silver	retail_catalog.retail_analytics.*	silver_customers, silver_products, silver_sales, dim_date	10K + 1K + 199,994 + 1,096	Cleaned, typed, deduplicated, quality-checked
Gold	retail_catalog.retail_analytics.*	4 Gold tables	366 + 10K + 7 + 1	Pre-aggregated business metrics for Power BI
Rejected	retail_catalog.retail_analytics.*	rejected_rows	15	DQ failure quarantine — preserved for investigation
Audit	retail_catalog.retail_analytics.*	pipeline_audit_log	Grows per run	Full operational run history with row counts and status

2.3 Technology Stack

Component	Technology	Details
Data Platform	Databricks	Unity Catalog enabled, SQL Warehouses for all queries
Storage Format	Delta Lake	ACID transactions, schema enforcement, time travel, data skipping
File Storage	Unity Catalog Volumes	Governed replacement for DBFS — full audit trail
Query Language	Spark SQL / Databricks SQL	100% SQL — no PySpark used in this project
Orchestration	Databricks Workflows	3-task job with dependency enforcement and CRON schedule
Visualisation	Microsoft Power BI	Desktop + Service, Import mode, DAX, scheduled daily refresh
Data Modelling	Star Schema	fact_sales + dim_customers + dim_products + dim_date

3. Data Modelling

3.1 Entity Relationship Diagram

The data model implements a Star Schema with surrogate keys and SCD Type 2 tracking columns for customer and product dimensions. This design supports both current-state analytics and historical analysis of how customer and product attributes changed over time.

3.2 Table Definitions

dim_customer — Customer Dimension (SCD Type 2)

Column	Type	Description
customer_key	INT PK	Surrogate key — auto-generated, unique per version
customer_id	STRING	Natural/business key from source system
customer_name	STRING	Customer full name
effective_start_date	DATE	Date this version of the record became active
effective_end_date	DATE	Date this version expired (NULL if current)
is_current_flag	BOOLEAN	TRUE if this is the active/current record version
load_timestamp	TIMESTAMP	When this record was loaded into the warehouse

dim_product — Product Dimension

Column	Type	Description
product_key	INT PK	Surrogate key auto-generated
product_id	STRING	Natural/business key from source
product_name	STRING	Product display name
category	STRING	Product category (Beauty, Electronics, etc.)
effective_start_date	DATE	Version start date
effective_end_date	DATE	Version end date (NULL if current)
is_current_flag	BOOLEAN	TRUE if active version
load_timestamp	TIMESTAMP	Load timestamp

dim_date — Date Dimension

Column	Type	Description
date_key	INT PK	YYYYMMDD integer key — fast join from fact table
full_date	DATE	Full calendar date
day	INT	Day of month (1–31)
month	INT	Month number (1–12)
month_name	STRING	Full month name (January–December)
quarter	INT	Calendar quarter (1–4)
year	INT	Calendar year
weekend_flag	BOOLEAN	TRUE if Saturday or Sunday

fact_sales — Central Fact Table

Column	Type	Description
sales_key	INT PK	Surrogate key for the fact record
order_id	STRING	Natural business key (sale_id from source)
customer_key	INT FK	Foreign key → dim_customer.customer_key
product_key	INT FK	Foreign key → dim_product.product_key
date_key	INT FK	Foreign key → dim_date.date_key
quantity	INT	Units purchased in this transaction
unit_price	DECIMAL	Product price at time of sale
revenue	DECIMAL	quantity × unit_price — pre-computed line revenue
load_timestamp	TIMESTAMP	When the record was loaded
record_source	STRING	Source file/batch that produced this record

4. Source Data & Exploratory Data Analysis

4.1 Source Datasets

File	Rows	Columns	Description
Master_customers.csv	10,009	5	customer_id, full_name, email, city, signup_date
master_products.csv	1,000	4	product_id, product_name, category, price
sales.csv	200,000	9	sale_id, event_timestamp, customer_id, product_id, quantity, currency, channel, source_file, ingestion_timestamp

4.2 Data Issues Identified in EDA

#	Dataset	Issue	Rows	Severity	Resolution in Silver
1	Customers	9 fully duplicate rows (IDs 547–551, 652–655)	9	**HIGH**	ROW_NUMBER() OVER (PARTITION BY id) — keep rn=1
2	Customers	signup_date stored as STRING	10,009	**MEDIUM**	TRY_TO_TIMESTAMP(col, 'dd-MM-yyyy')
3	Customers	9 NULL signup_date values	9	**MEDIUM**	TRY_TO_TIMESTAMP returns NULL gracefully — logged to rejected_rows
4	Products	price stored as STRING (corrupted values present)	1,000	**HIGH**	TRY_CAST with RLIKE guard — non-numeric → NULL + price_flag
5	Products	Corrupted prices: '356t78' and 'pp567'	2	**HIGH**	RLIKE '^[0-9]+(\.[0-9]+)?$' — non-matching → NULL
6	Products	3 products with price = 0	3	**MEDIUM**	Retained, flagged 'zero_price', logged to rejected_rows
7	Sales	customer_id / product_id as FLOAT64 (NULLs)	200,000	**MEDIUM**	TRY_CAST after NULL/zero filter → clean INT
8	Sales	1 NULL customer_id, 1 NULL product_id	2	**HIGH**	WHERE TRY_CAST IS NOT NULL
9	Sales	2 rows customer_id=0, 2 rows product_id=0	4	**HIGH**	WHERE TRY_CAST > 0
10	Sales	quantity contains 'b' and 'd' (alphabetic)	2	**HIGH**	CASE WHEN RLIKE '^[0-9]+$' THEN TRY_CAST ELSE NULL END
11	Sales	Timestamps stored as STRING	200,000	**MEDIUM**	TRY_TO_TIMESTAMP(col, 'dd-MM-yyyy HH:mm')
12	Sales	sales_batch_5.csv has only 1 record	1	**MEDIUM**	All batches read together — row count logged in audit

5. Bronze Layer — Raw Ingestion

5.1 Volume Setup

```
CREATE CATALOG  IF NOT EXISTS retail_catalog
  COMMENT 'Retail analytics lakehouse';
CREATE SCHEMA   IF NOT EXISTS retail_catalog.retail_analytics;
CREATE VOLUME   IF NOT EXISTS retail_catalog.retail_analytics.retail_data
  COMMENT 'CSV file landing zone';
```

5.2 bronze_sales

```
CREATE OR REPLACE TABLE retail_catalog.retail_analytics.bronze_sales
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
SELECT *,
  'Volume:retail_data/sales.csv' AS _source_file,
  current_timestamp()            AS _ingestion_timestamp,
  'bronze_ingestion_v1'          AS _pipeline_version
FROM read_files(
  '/Volumes/retail_catalog/retail_analytics/retail_data/sales.csv',
  format => 'csv', header => true, inferSchema => false);
```

5.3 bronze_master

```
CREATE OR REPLACE TABLE retail_catalog.retail_analytics.bronze_master
USING DELTA AS
SELECT 'customers' AS _record_type, customer_id AS id, full_name AS name,
  email AS email_or_category, city AS city_or_price, signup_date AS extra_field,
  NULL AS extra_field2, current_timestamp() AS _ingestion_timestamp, ...
FROM read_files('.../Master_customers.csv', format=>'csv', header=>true,
inferSchema=>false)
UNION ALL
SELECT 'products' AS _record_type, product_id AS id, product_name AS name,
  category AS email_or_category, price AS city_or_price, NULL AS extra_field, ...
FROM read_files('.../master_products.csv', format=>'csv', header=>true,
inferSchema=>false);
```

6. Silver Layer — Cleaning & Typing

6.1 silver_customers

```
CREATE OR REPLACE TABLE retail_catalog.retail_analytics.silver_customers USING DELTA AS
WITH deduped AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY id ORDER BY _ingestion_timestamp DESC) AS rn
  FROM retail_catalog.retail_analytics.bronze_master WHERE _record_type = 'customers'
)
SELECT
  CAST(id AS INT)                              AS customer_id,
  TRIM(name)                                   AS full_name,
  LOWER(TRIM(email_or_category))               AS email,
  TRIM(city_or_price)                          AS city,
  TRY_TO_TIMESTAMP(extra_field,'dd-MM-yyyy')   AS signup_date
FROM deduped WHERE rn = 1;
```

6.2 silver_products -- TRY_CAST price fix

```
CASE
  WHEN city_or_price RLIKE '^[0-9]+(\.[0-9]+)?$'
    THEN TRY_CAST(city_or_price AS DECIMAL(12,2))
  ELSE NULL
END AS unit_price,
CASE
  WHEN TRY_CAST(city_or_price AS DECIMAL(12,2)) IS NULL THEN 'corrupted_price'
  WHEN TRY_CAST(city_or_price AS DECIMAL(12,2)) = 0    THEN 'zero_price'
  ELSE 'valid'
END AS price_flag
```

6.3 silver_sales

```
CREATE OR REPLACE TABLE retail_catalog.retail_analytics.silver_sales
USING DELTA PARTITIONED BY (sale_year) AS
SELECT
  TRY_CAST(sale_id     AS INT)                                    AS sale_id,
  TRY_TO_TIMESTAMP(event_timestamp, 'dd-MM-yyyy HH:mm')           AS event_timestamp,
  DATE(TRY_TO_TIMESTAMP(event_timestamp, 'dd-MM-yyyy HH:mm'))     AS sale_date,
  YEAR(TRY_TO_TIMESTAMP(event_timestamp, 'dd-MM-yyyy HH:mm'))     AS sale_year,
  TRY_CAST(customer_id AS INT)                                    AS customer_id,
  TRY_CAST(product_id  AS INT)                                    AS product_id,
  CASE WHEN quantity RLIKE '^[0-9]+$' THEN TRY_CAST(quantity AS INT) ELSE NULL END AS
quantity,
  UPPER(TRIM(currency)) AS currency,  LOWER(TRIM(channel)) AS channel
FROM retail_catalog.retail_analytics.bronze_sales
WHERE TRY_CAST(customer_id AS INT) IS NOT NULL AND TRY_CAST(customer_id AS INT) > 0
  AND TRY_CAST(product_id  AS INT) IS NOT NULL AND TRY_CAST(product_id  AS INT) > 0;
```

6.4 dim_date

```sql
CREATE OR REPLACE TABLE retail_catalog.retail_analytics.dim_date AS
WITH date_range AS (
  SELECT EXPLODE(SEQUENCE(DATE '2022-01-01', DATE '2024-12-31', INTERVAL 1 DAY)) AS date
)
SELECT date, DAY(date) AS day, MONTH(date) AS month,
  DATE_FORMAT(date,'MMMM') AS month_name, QUARTER(date) AS quarter, YEAR(date) AS year,
  WEEKOFYEAR(date) AS week_of_year,
  CASE WHEN DAYOFWEEK(date) IN (1,7) THEN TRUE ELSE FALSE END AS is_weekend,
  CAST(DATE_FORMAT(date,'yyyyMMdd') AS INT) AS date_key
FROM date_range;
```

7. Gold Layer — Business Aggregations

7.1 fact_sales — Star Schema

```
CREATE OR REPLACE TABLE retail_catalog.retail_analytics.fact_sales
USING DELTA PARTITIONED BY (sale_year)
TBLPROPERTIES ('delta.
autoOptimize.optimizeWrite'='true','delta.autoOptimize.autoCompact'='true')
AS
SELECT s.sale_id, s.event_timestamp, s.sale_date, s.sale_year,
  dd.date_key, dd.month, dd.month_name, dd.quarter, dd.week_of_year, dd.is_weekend,
  s.customer_id, c.full_name AS customer_name, c.city, c.signup_date,
  s.product_id, p.product_name, p.category, p.unit_price, s.quantity,
  ROUND(s.quantity * p.unit_price, 2) AS line_revenue,
  s.currency, s.channel
FROM silver_sales s
INNER JOIN dim_customers c  ON s.customer_id = c.customer_id
INNER JOIN dim_products  p  ON s.product_id  = p.product_id
LEFT  JOIN dim_date      dd ON s.sale_date   = dd.date;
```

7.2 Gold Tables Summary

Gold Table	Rows	Partition	Key Metric
gold_daily_sales	366	year	total_revenue, total_transactions, total_items_sold, avg_transaction_value
gold_customer_lifetime_value	10,000	city	lifetime_revenue, total_orders, avg_order_value, first/last purchase dates
gold_category_performance	7	category	category_revenue, items_sold, category_share_pct
gold_repeat_customer_rate	1	None	repeat_customers, first_time_buyers, repeat_customer_rate_pct

8. Performance Optimisation

```
OPTIMIZE retail_catalog.retail_analytics.fact_sales ZORDER BY (sale_date, customer_id,
product_id);
OPTIMIZE retail_catalog.retail_analytics.gold_daily_sales ZORDER BY (sale_date);
OPTIMIZE retail_catalog.retail_analytics.gold_customer_lifetime_value ZORDER BY
(customer_id);


VACUUM retail_catalog.retail_analytics.fact_sales    RETAIN 168 HOURS;
VACUUM retail_catalog.retail_analytics.silver_sales  RETAIN 168 HOURS;


ANALYZE TABLE retail_catalog.retail_analytics.fact_sales  COMPUTE STATISTICS FOR ALL
COLUMNS;
ANALYZE TABLE retail_catalog.retail_analytics.silver_sales COMPUTE STATISTICS FOR ALL
COLUMNS;
```

Table	Partition Column	Reason
silver_sales / fact_sales	sale_year	Year-scoped queries skip irrelevant partitions entirely
gold_daily_sales	year	Power BI date slicers filter by year — major query speedup
gold_customer_lifetime_value	city	City is a common filter in customer analytics
gold_category_performance	category	Each category queried independently in reports
silver_customers, dim_date, etc.	None	Small tables — partitioning overhead would hurt not help

9. Automated Workflow & Audit Logging

9.1 Databricks Job Configuration

Task	Notebook	Depends On	Purpose
1 — bronze_ingestion	01_bronze_ingestion	None (start)	Ingest CSVs → Delta Bronze tables
2 — silver_transformation	02_silver_transformation	bronze_ingestion	Clean, type, deduplicate → Silver + DQ
3 — gold_aggregations	03_gold_aggregations	silver_transformation	Aggregate → Gold tables + OPTIMIZE

Schedule: Daily at 02:00 AM IST · CRON: `0 0 2 * * ?` (Quartz format — 6 fields) · Max retries: 2 · Retry delay: 5 min

9.2 Validated Audit Log — Final Pipeline Run

Target Table	Rows Written	Rows Rejected	Status
bronze_sales	200,000	0	✓ SUCCESS
bronze_master	11,009	0	✓ SUCCESS
silver_customers	10,000	0	✓ SUCCESS
silver_products	1,000	0	✓ SUCCESS
silver_sales	199,994	15	✓ SUCCESS
dim_date	1,096	0	✓ SUCCESS
gold_daily_sales	366	0	✓ SUCCESS
gold_customer_lifetime_value	10,000	0	✓ SUCCESS
gold_category_performance	7	0	✓ SUCCESS
gold_repeat_customer_rate	1	0	✓ SUCCESS

9.3 Incremental Load — MERGE Pattern

```
MERGE INTO retail_catalog.retail_analytics.silver_sales AS target
USING ( SELECT TRY_CAST(sale_id AS INT) AS sale_id, ... FROM bronze_sales
        WHERE TRY_CAST(customer_id AS INT) IS NOT NULL
          AND TRY_CAST(customer_id AS INT) > 0 ) AS source
ON target.sale_id = source.sale_id
WHEN MATCHED AND target.event_timestamp != source.event_timestamp THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

10. Power BI Dashboard

10.1 Dashboard Pages & Visuals

Page	Key Visuals	Source Tables
Executive Summary	5 KPI cards, Daily Revenue line chart, Revenue by Channel donut, Heatmap matrix, Repeat vs First-time buyers	gold_daily_sales, gold_repeat_customer_rate
Sales Deep Dive	Monthly Revenue vs Prior Year, Weekend vs Weekday stacked bar, Revenue by Quarter waterfall, Daily transactions scatter	gold_daily_sales, dim_date
Product & Category	Category Revenue bar chart, Share donut, Revenue treemap, Items vs Revenue scatter, Category performance table	gold_category_performance
Customer Intelligence	4 KPI cards, LTV distribution histogram, Top 20 Customers bar, Customer Loyalty scatter, Revenue by city, Customer detail table	gold_customer_lifetime_value, gold_repeat_customer_rate

10.2 Key DAX Measures

Measure	DAX	Purpose
Total Revenue	SUM(gold_daily_sales[total_revenue])	Base revenue
Revenue MTD	CALCULATE([Total Revenue], DATESMTD(dim_date[date]))	Month-to-date
Revenue YTD	CALCULATE([Total Revenue], DATESYTD(dim_date[date]))	Year-to-date
Revenue vs Prior Month	DIVIDE(Current - Prior, Prior, 0) via DATEADD	MoM % change
Avg Transaction Value	DIVIDE([Total Revenue], [Total Transactions], 0)	Revenue per sale
Repeat Customer Rate	MAX(gold_repeat_customer_rate[repeat_customer_rate_pct])	Loyalty KPI
Revenue Trend Colour	IF([Revenue vs Prior Month] >= 0, '#1B7A34', '#C00000')	Conditional format

10.3 Model Relationships

From	Column	To	Column	Type
fact_sales	customer_id	dim_customers	customer_id	Many-to-One ✓
fact_sales	product_id	dim_products	product_id	Many-to-One ✓
fact_sales	sale_date	dim_date	date	Many-to-One ✓
gold_daily_sales	sale_date	dim_date	date	Many-to-One ✓
gold_category_performance	—	Standalone	—	No relationship — pre-aggregated Gold table

11. Errors Encountered & Resolutions

Error	Root Cause	Exact Fix
[CAST_INVALID_INPUT] 'NA' → INT	Spark does not short-circuit CAST in WHERE/CASE — evaluates all rows	Replace every CAST with TRY_CAST in Silver layer
[CAST_INVALID_INPUT] '356t78' → DECIMAL	Same optimiser issue — RLIKE guard does not prevent CAST evaluation	Use TRY_CAST in price_flag CASE — returns NULL on bad input
[WRONG_NUM_ARGS] cast requires 0 params	CAST(col, 'format') is invalid Spark SQL — no format argument	Use DATE(TRY_TO_TIMESTAMP(col, format)) instead
[PARSE_SYNTAX_ERROR] near 'FIELD'	ADD PARTITION FIELD is Iceberg syntax — unsupported on Delta	Delta partitioning only at CREATE TABLE time — use CREATE OR REPLACE
[NOT_SUPPORTED] UNCACHE on SQL Warehouse	CACHE/UNCACHE only works on all-purpose / job clusters	Remove CACHE — SQL Warehouses use automatic disk caching
[UNSUPPORTED_SUBQUERY] in VALUES clause	Spark SQL does not allow subqueries inside VALUES(...)	Replace VALUES(...) with SELECT ... (subqueries work in SELECT)
[TABLE_OR_VIEW_NOT_FOUND] pipeline_start_silver	TEMP VIEW created in wrong cell or missing from notebook	Add CREATE OR REPLACE TEMP VIEW as the very first cell of each notebook

12. Technical Learnings

- Medallion Architecture — the Bronze/Silver/Gold separation is not just organisational; it is fundamental to data quality. Bronze preserves raw data. Silver applies all cleaning in one place. Gold serves business users without exposing complexity.
- TRY_CAST vs CAST — Spark SQL's Catalyst optimiser does not guarantee short-circuit evaluation. Any CAST on a Bronze column must use TRY_CAST to handle dirty values gracefully without crashing the pipeline.
- Unity Catalog Volumes are the modern governed replacement for DBFS. Same path interface but with full audit trail, access control, and Unity Catalog lineage.
- DATE() vs CAST(x AS DATE) — Spark SQL does not support a two-argument CAST for date formatting. Use DATE(timestamp_col) or TO_DATE(string_col, format).
- Quartz CRON uses 6 fields (seconds minutes hours day month day-of-week) and requires ? for either day-of-month or day-of-week. Different from standard Unix 5-field CRON.
- OPTIMIZE + ZORDER is the most impactful single performance optimisation for Delta fact tables. It co-locates related data and enables file skipping on date-filtered queries.
- MERGE INTO for incremental loads is the enterprise-standard pattern — full table rebuilds daily are expensive and unnecessary when only new records need to be added.
- Power BI pre-aggregated Gold tables (category_performance, repeat_rate) should be standalone with no relationships — their grain does not match the star schema and relationship errors result.
- SQL Warehouses vs job clusters — CACHE TABLE, certain DDL operations, and stored procedures behave differently. SQL Warehouses provide automatic caching — no manual CACHE statements needed or supported.
- EDA before pipeline development is non-negotiable — identifying all 12 data issues before writing transformations prevented every potential pipeline failure during development.

13. Future Improvements

Improvement	Description	Business Value
Real-Time Streaming	Kafka + Databricks Structured Streaming to process sales events in real time instead of daily batch	Live revenue dashboards, real-time fraud detection
SCD Type 2 Dimensions	Implement effective_start_date / effective_end_date / is_current_flag as shown in the data model diagram using MERGE logic	Historical analysis — correct customer attributes at time of sale
Surrogate Keys	Implement auto-incrementing surrogate keys (customer_key, product_key) in dimension tables matching the data model diagram	Better join performance, natural key decoupling
dbt Integration	Replace SQL notebooks with dbt models for version control, automated testing, and auto-generated documentation	Faster development, fewer bugs, self-documenting pipeline
Great Expectations	Automated DQ expectation suites run at ingestion time with data docs published to a shared portal	Proactive quality monitoring before issues reach Gold
ML Churn Prediction	Databricks MLflow + Feature Store — churn model trained on Gold layer customer data	Predict which customers will stop purchasing
CI/CD Pipeline	GitHub Actions to auto-deploy notebooks and run quality gate checks on every pull request	Enterprise deployment governance
Delta Live Tables	Migrate pipeline to DLT for declarative pipeline definitions with built-in lineage and monitoring UI	Reduced pipeline maintenance overhead

14. Complete Table Inventory & Row Count Validation

Table	Layer	Rows	Partition	Purpose
bronze_sales	Bronze	200,000	None	Raw sales CSV exact replica
bronze_master	Bronze	11,009	None	Raw customers + products combined
silver_customers	Silver	10,000	None	Deduplicated, typed customers
silver_products	Silver	1,000	None	Type-cast products with price_flag
silver_sales	Silver	199,994	sale_year	Cleaned, filtered sales fact
dim_date	Silver	1,096	None	Generated date dimension 2022–2024
dim_customers	Dim	10,000	None	Customer dimension (star schema)
dim_products	Dim	1,000	None	Product dimension (star schema)
fact_sales	Fact	~199,994	sale_year	Central star schema fact table
gold_daily_sales	Gold	366	year	Daily revenue aggregations
gold_customer_lifetime_value	Gold	10,000	city	Per-customer LTV metrics
gold_category_performance	Gold	7	category	Revenue per product category
gold_repeat_customer_rate	Gold	1	None	Customer loyalty summary row
rejected_rows	Quality	15	None	DQ failures — quarantine table
pipeline_audit_log	Ops	Growing	None	Full pipeline run history

14.1 Row Count Transitions Explained

- 200,000 raw → 199,994 silver_sales: 6 rows removed (1 NULL customer_id + 2 zero customer_ids + 1 NULL product_id + 2 zero product_ids)
- 10,009 raw → 10,000 silver_customers: 9 fully duplicate rows removed by ROW_NUMBER() deduplication
- 1,000 products → 1,000 silver_products: all rows retained; 5 flagged but not dropped (2 corrupted price, 3 zero price)
- 366 gold_daily_sales rows = 366 distinct sale dates between Jan 1 2023 and Jan 1 2024 inclusive
- 1,096 dim_date rows = 365 (2022) + 365 (2023) + 366 (2024, leap year) = 1,096 calendar dates
- 15 rejected_rows = 2 invalid quantity + 2 corrupted price + 3 zero price + ~5 invalid email + ~3 null signup_date